

DTU



lambda
D A λ S

28-29 JULY 2022
KRAKÓW | POLAND

Teaching Functional Programmers Logic and Metatheory

Frederik Krogsdal Jacobsen Jørgen Villadsen

Technical University of Denmark

- We want to teach our students logic
- Pen and paper proofs are good, but the connections to the real world are not always clear
- Textbooks sometimes sweep complexity under the rug
- Functional programming (in Isabelle/HOL) to the rescue!

Our approach

- Define every concept as a functional program
- Use a proof assistant (a functional language with a *very* strong type system) to prove any theorems
- Students can still read the informal definitions and proofs. . .
- . . . but they can consult the formalization if they need more details
- Implementations allow experiments and easy modifications
- The proof assistant makes sure we didn't forget anything in our proofs
- Isabelle/HOL allows export to languages such as Haskell and Scala

Our contributions

- Functional implementations of several logical systems
- Formally verified proofs of common properties
- Evaluation of the usefulness of our approach

Basics

Syntax

Proposition $P, Q, R, S, \dots$

Falsity $\perp$

Implication $P \rightarrow Q$

Semantics

We have an interpretation i , which assigns truth values to propositions

Proposition: Look up P in i

Falsity: Always false

Implication: If P is true, then Q must also be

What do we want to find out?

Satisfiability: is it possible to choose an i that makes the formula true?

Validity: is the formula true no matter what i we choose?

We can decide this by trying all possible options, but the number grows exponentially

Sequent calculus

- Assumptions $\vdash$ Goals

$$\frac{}{\Gamma \cup \{p\} \vdash \Delta \cup \{p\}} \text{ BASIC}$$

$$\frac{}{\Gamma \cup \{\perp\} \vdash \Delta} \text{ FALSITY-LEFT}$$

$$\frac{\Gamma \vdash \Delta \cup \{p\} \quad \Gamma \cup \{q\} \vdash \Delta}{\Gamma \cup \{p \rightarrow q\} \vdash \Delta} \text{ IMPLICATION-LEFT}$$

$$\frac{\Gamma \cup \{p\} \vdash \Delta \cup \{q\}}{\Gamma \vdash \Delta \cup \{p \rightarrow q\}} \text{ IMPLICATION-RIGHT}$$

Properties of the calculus

Soundness: if we can construct a proof using the rules, the formula is valid

Completeness: if the formula is valid, we can construct a proof using the rules

Properties of the calculus

Soundness: if we can construct a proof using the rules, the formula is valid

Completeness: if the formula is valid, we can construct a proof using the rules

Proof

By induction.

It's never that easy

$$\frac{}{\Gamma \cup \{p\} \vdash \Delta \cup \{p\}} \text{ BASIC}$$

$$\frac{}{\Gamma \cup \{\perp\} \vdash \Delta} \text{ FALSITY-LEFT}$$

$$\frac{\Gamma \vdash \Delta \cup \{p\} \quad \Gamma \cup \{q\} \vdash \Delta}{\Gamma \cup \{p \rightarrow q\} \vdash \Delta} \text{ IMPLICATION-LEFT}$$

$$\frac{\Gamma \cup \{p\} \vdash \Delta \cup \{q\}}{\Gamma \vdash \Delta \cup \{p \rightarrow q\}} \text{ IMPLICATION-RIGHT}$$

Why does the process of constructing a proof terminate exactly when the formula is valid?

Let's make a computer check our work

datatype *'a form*

= Pro 'a (⟨ · ⟩) | Falsity (⟨ ⊥ ⟩) | Imp ⟨ 'a form ⟩ ⟨ 'a form ⟩ (infixr ⟨ → ⟩ 0)

primrec *semantics* **where**

⟨ semantics i (·n) = i n ⟩ |

⟨ semantics - ⊥ = False ⟩ |

⟨ semantics i (p → q) = (semantics i p → semantics i q) ⟩

abbreviation *⟨ sc X Y i ≡ (∀ p ∈ set X. semantics i p) → (∃ q ∈ set Y. semantics i q) ⟩*

Let's make a computer check our work

primrec *member* where

$\langle \text{member} - [] = \text{False} \rangle \mid$
 $\langle \text{member } m (n \# A) = (m = n \vee \text{member } m A) \rangle$

lemma *member-iff* [iff]: $\langle \text{member } m A \longleftrightarrow m \in \text{set } A \rangle$
by (induct A) simp-all

primrec *common* where

$\langle \text{common} - [] = \text{False} \rangle \mid$
 $\langle \text{common } A (m \# B) = (\text{member } m A \vee \text{common } A B) \rangle$

lemma *common-iff* [iff]: $\langle \text{common } A B \longleftrightarrow \text{set } A \cap \text{set } B \neq \{\} \rangle$
by (induct B) simp-all

Let's make a computer check our work

function *mp* where

$$\begin{aligned} &\langle mp\ A\ B\ (\cdot n\ \# C)\ [] = mp\ (n\ \# A)\ B\ C\ [] \rangle | \\ &\langle mp\ A\ B\ C\ (\cdot n\ \# D) = mp\ A\ (n\ \# B)\ C\ D \rangle | \\ &\langle mp\ -\ (\perp\ \# -)\ [] = True \rangle | \\ &\langle mp\ A\ B\ C\ (\perp\ \# D) = mp\ A\ B\ C\ D \rangle | \\ &\langle mp\ A\ B\ ((p \rightarrow q)\ \# C)\ [] = (mp\ A\ B\ C\ [p] \wedge mp\ A\ B\ (q\ \# C)\ []) \rangle | \\ &\langle mp\ A\ B\ C\ ((p \rightarrow q)\ \# D) = mp\ A\ B\ (p\ \# C)\ (q\ \# D) \rangle | \\ &\langle mp\ A\ B\ []\ [] = common\ A\ B \rangle \end{aligned}$$

by *pat-completeness simp-all*

termination

by (*relation* $\langle measure\ (\lambda(-, -, C, D). \sum p \leftarrow C @ D. size\ p) \rangle$) *simp-all*

theorem *main*: $\langle (\forall i. sc\ (map\ \cdot\ A\ @\ C)\ (map\ \cdot\ B\ @\ D)\ i) \longleftrightarrow mp\ A\ B\ C\ D \rangle$

by (*induct rule*: *mp.induct*) (*simp-all, blast, meson, fast*)

Let's make a computer check our work

definition $\langle \text{prover } p \equiv mp \text{ [] [] [] } [p] \rangle$

corollary $\langle \text{prover } p \longleftrightarrow (\forall i. \text{ semantics } i \ p) \rangle$
unfolding prover-def by (*simp flip: main*)

The course

- MSc level course with functional programming prerequisites
- Many students have never seen formal logic before
- Lectures, exercise sessions (with TAs), and assignments
- Exam is all programming and proving in Isabelle/HOL
- 43 students after the first few weeks

Weeks	Topics
1 – 2	Basic set theory, propositional logic, sequent calculus, automatic theorem proving
3 – 4	Syntax and semantics of first-order logic, natural deduction, the LCF approach
5 – 6	Isar, intuitionistic logic, foundational systems
7 – 8	Proof by contradiction, classical logic, higher-order logic, type theory
9 – 10	Proofs in sequent calculus
11 – 13	Metatheory, prover algorithms, program verification

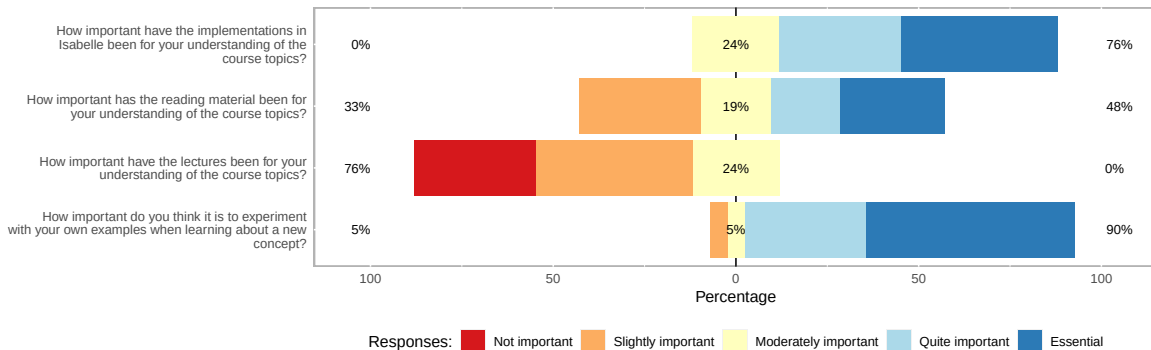
Survey design

- Anonymous survey of our 43 students
- 21 answers (48.8% response rate, 15.5% margin of error)
- 6 hypotheses
- 12 questions

Hypotheses

- 1 Concrete implementations in a programming language aid understanding of concepts in logic.
- 2 Students experiment with definitions to gain understanding.
- 3 Our formalizations make it clear to students how to implement the concepts in practice.
- 4 Our course makes students able to design and implement their own logical systems.
- 5 Prior experience with functional programming is useful for our course.
- 6 Our course helps students gain proficiency in functional programming.

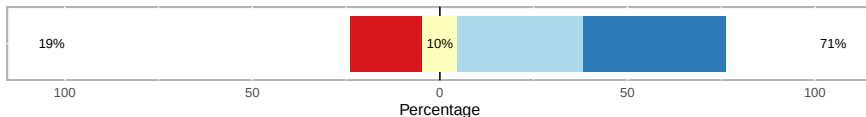
Results



Plausible: Concrete implementations in a programming language aid understanding of concepts in logic

Results

When using Isabelle, how often do you evaluate your own concrete examples to understand new concepts? (E.g. using the "value" command.)



Responses: ■ Almost never ■ Once in a while ■ Sometimes ■ Often ■ Almost always

Confirmed: Students experiment with definitions to gain understanding

Results

Rejected: Our formalizations make it clear to students how to implement the concepts in practice

Results

Rejected: Our course makes students able to design and implement their own logical systems

Results

Confirmed: Prior experience with functional programming is useful for our course (small to moderate association)

Results

Confirmed: Our course helps students gain proficiency in functional programming (large positive effect)

Interesting trends

Warning: Post-hoc analysis!

- It seems that students who think experimentation is more important do it less in Isabelle
- Students who were not confident functional programmers at the end were less confident that they could implement systems
- Students do not seem to get elevated past a basic understanding of functional programming
- Advanced concepts in functional programming do not seem to be needed

Open questions

- Why do students who think experimentation is important seem to do it less? Do they do it on paper instead?
- Does functional programming experience play a significant role in understanding of how to implement concepts in practice?
- Does functional programming experience play a significant role in understanding of how to design and implement one's own logical systems?
- Does our course have a positive effect on functional programming skill for students who are already confident functional programmers?

Ideas for improving our course

- Reduce time spent on lectures
- Add a project-based assignment
- Consider how to teach students how to design their own systems

Future surveys

- Larger sample sizes
- Pursue open questions
- Add information on grades and demographics
- Reduce self-selection bias
- Comparisons to pen-and-paper courses

Conclusion

- Our approach seems promising, but it needs more work
- Students find it hard to implement their own designs
- There are lots of opportunities for further research

More information:

Isabelle <https://isabelle.systems/>

Our paper <https://dx.doi.org/10.4204/EPTCS.363.5>

Related papers <https://people.compute.dtu.dk/fkjac>